

RememberME – Relatório final

Instituto do Emprego e da Formação Profissional, I.P.

Centro de Emprego e Formação Profissional do Seixal

Projeto Final: Desenvolvimento Colaborativo de Solução Tecnológica – RememberME

IEFP, Programação (Nível 5) · Formador: João Carlos Cid Ferreira · Setembro de 2026

Versão entregue: v1.0 · Repositório: github.com/RenatoMoraesDev/RememberME

Equipa:

- Felipe Ribeiro
- Niley Barros
- Renato Moraes

Sumário

1. Identificação do projeto e dos elementos.....	4
2. Enquadramento e problema.....	4
3. Objetivos.....	5
4. Requisitos definidos.....	5
4.1 Requisitos funcionais.....	5
4.2 Requisitos não funcionais.....	6
5. Tecnologias utilizadas.....	7
6. Arquitetura da solução.....	8
6.1 Como as peças funcionam em conjunto.....	9
6.2 Dados.....	9
6.3 Decisões de desenho.....	9
7. Organização e distribuição do trabalho.....	10
8. Descrição das funcionalidades implementadas.....	11
8.1 Lembretes a hora fixa (RF01).....	11
8.2 Lembretes recorrentes numa janela (RF02).....	11
8.3 Gestão de lembretes (RF06).....	11
8.4 Serviço na bandeja (RF03).....	11
8.5 Persistência local (RF04).....	11
8.6 Tipos de aviso (RF07, parcial).....	11
8.7 Interface gráfica (RF10, parcial).....	11
8.8 Distribuição e arranque automático (RF08).....	11
9. Estratégia de utilização de Git e GitHub.....	12
9.1 Branches.....	12
9.2 Commits.....	12
9.3 Pull Requests.....	12
9.4 Revisão de código.....	12
9.5 Versões.....	12
9.6 Documentação viva.....	12
10. Testes realizados.....	13
11. Dificuldades encontradas.....	14
12. Soluções adotadas.....	14
13. Limitações.....	15
14. Melhorias futuras.....	15
15. Conclusão.....	16
16. Apreciação do contributo de cada elemento.....	16

16.1 Felipe Ribeiro.....	16
16.2 Niley Barros.....	17
16.3 Renato Moraes.....	18
16.4 Apreciação global.....	18

1. Identificação do projeto e dos elementos

O RememberME é uma aplicação para Windows que corre em segundo plano e lembra o formador de tarefas administrativas que se repetem durante a aula, como fechar a sessão ou fazer a chamada. Os avisos são configurados para uma hora fixa ou para se repetirem dentro de um intervalo de horário.

Elemento	Conta GitHub	Papel principal
Felipe Ribeiro	felipe-g-ribeiro	Linha de comandos, interface gráfica, arranque automático
Niley Barros	sacramentoniley-hub e nileysacramento	Notificações e tipos de ação, execução e registo dos testes, quadro de tarefas
Renato Moraes	RenatoMoraesDev e hexemeister	Coordenação, arquitetura, núcleo da aplicação, empacotamento, documentação

O Niley e o Renato fizeram commits a partir de duas contas cada um, por isso aparecem duas identidades por pessoa no histórico. Os números da secção 16 já somam as duas.

A versão final está publicada como [release v1.0](#), com o executável portátil e o instalador prontos a descarregar.

2. Enquadramento e problema

Numa aula, a atenção do formador está no conteúdo e nos formandos. As tarefas administrativas pequenas, como fechar a sessão de acesso à hora certa ou registar as presenças, dependem de ele se lembrar delas sozinho, e é aí que falham. Nenhuma é difícil. O problema é a hora a que têm de ser feitas, sempre a meio de outra coisa.

Quem sofre o esquecimento é o formador, mas as consequências chegam à instituição, que fica sem o registo de presenças ou de sessão no momento em que devia existir. O que o grupo propôs foi tirar essa carga da memória do formador: um programa discreto, que não pede atenção nenhuma até à hora marcada, e que nessa altura avisa.

O público-alvo são os formadores do IEFP e, de forma mais geral, qualquer formador com rotinas administrativas presas a um horário.

3. Objetivos

O objetivo principal era reduzir o esquecimento de tarefas administrativas recorrentes durante a aula, através de lembretes automáticos e configuráveis.

Na Etapa 1, esse objetivo foi desdobrado em objetivos específicos:

- criar lembretes a uma hora fixa ou recorrentes dentro de uma janela de horário (por exemplo, de hora a hora entre às 8h e às 14h);
- avisar por vários canais: notificação do sistema, som, janela no ecrã ou abertura de uma aplicação ou página web;
- funcionar em segundo plano sem interferir com o resto do trabalho no computador;
- arrancar automaticamente com o sistema operativo;
- escrever código portátil para Windows, macOS e Linux, testando e empacotando apenas em Windows, que era o único sistema a que o grupo tinha acesso.

Como o grupo dispunha de 32 horas de aula para as cinco etapas, os requisitos foram separados logo à partida em dois grupos: um núcleo obrigatório (MVP) e um conjunto de requisitos incrementais que só avançariam se sobrasse tempo.

4. Requisitos definidos

4.1 Requisitos funcionais

A tabela mostra o que foi definido na Etapa 1 e o estado de cada requisito na versão entregue.

ID	Requisito	Tipo	Estado na v1.0
RF01	Notificação a uma hora fixa	MVP	Implementado
RF02	Recorrência dentro de uma janela de horário	MVP	Implementado, com limitações nos intervalos (ver secção 13)
RF03	Ícone na bandeja do sistema com menu	MVP	Parcial: menu com “Abrir” e “Sair”, sem a lista de lembretes ativos
RF04	Persistência local, sem rede	MVP	Implementado (SQLite)

ID	Requisito	Tipo	Estado na v1.0
RF05	Lembretes administrativos pré-carregados	MVP	Não implementado
RF06	Adicionar, editar e remover lembretes	MVP	Implementado na linha de comandos e na interface gráfica
RF07	Vários tipos de ação por lembrete	Incremental	Parcial: a lógica existe, mas o utilizador só escolhe o pop-up (na GUI)
RF08	Arranque com o sistema operativo	Incremental	Implementado, como opção do instalador
RF09	Execução de um comando arbitrário	Incremental	Não implementado
RF10	Interface gráfica	Incremental	Parcial, em Tkinter em vez do PySide6 previsto

4.2 Requisitos não funcionais

ID	Requisito	Situação
RNF01	Código multiplataforma; testes e empacotamento só em Windows	Cumprido no núcleo. A GUI e o som usam funções exclusivas do Windows
RNF02	Baixo consumo de recursos em uso contínuo	Sem medição formal; o processo residente passa quase todo o tempo parado à espera do próximo disparo
RNF03	Configuração sem conhecimentos técnicos	Coberto pela interface gráfica e pelo instalador
RNF04	Dados locais, sem internet	Cumprido

ID	Requisito	Situação
RNF05	Confirmação e registo ao executar comandos	Não se aplica, porque o RF09 não foi implementado

5. Tecnologias utilizadas

Área	Tecnologia	Motivo da escolha
Linguagem	Python 3.11+	Adequada a ferramentas de linha de comandos e desktop, e corre nos três sistemas operativos
Linha de comandos	Typer	Gera a ajuda e valida as opções a partir das próprias funções
Agendamento	APScheduler	Os disparos ficam presos ao relógio e a janela de horário cabe no próprio gatilho
Bandeja do sistema	pystray	Um só código para o ícone e o menu nos três sistemas
Notificações	plyer	Notificação nativa do sistema com uma única chamada
Base de dados	SQLite	Já vem com o Python, guarda tudo num ficheiro local e não precisa de servidor
Interface gráfica	Tkinter	Também vem com o Python e não acrescenta peso ao executável
Ambiente e dependências	uv	Um comando prepara o ambiente e o ficheiro <code>uv.lock</code> garante versões iguais nas três máquinas

Área	Tecnologia	Motivo da escolha
Empacotamento	PyInstaller e Inno Setup	O primeiro gera o executável; o segundo gera o instalador para Windows
Diagramas	D2	Diagramas escritos em texto, versionados como o código
Colaboração	Git, GitHub e VS Code	Controlo de versões, Pull Requests e releases

A escolha do motor de agendamento merece uma nota, porque foi a única decidida com código a correr e não apenas com documentação. Comparámos duas bibliotecas, `schedule` e `APScheduler`. A primeira conta os intervalos a partir do momento em que o programa arranca: se o computador for ligado às 08:07, um lembrete de hora a hora dispara às 09:07, 10:07 e assim por diante, e muda a cada reinício. O `APScheduler` dispara às 08:00, 09:00, 10:00, faça-se o arranque quando se fizer. Para lembretes que acompanham o horário de uma aula, esse comportamento decidiu a escolha.

O PySide6 estava previsto para a interface gráfica. A GUI acabou por ser feita em Tkinter. Além do Tkinter já vir com o Python e manter o executável pequeno, houve problemas na implementação com o PySide6, o que levou a sua substituição.

6. Arquitetura da solução

O código está em `src/rememberme/` e divide-se em dez ficheiros Python, cerca de 1 200 linhas no total. Numa aplicação desktop sem servidor, a divisão clássica em frontend, backend e dados fica assim:

Camada	Módulo	Função
Interface	<code>cli.py</code>	Comandos <code>add</code> , <code>list</code> , <code>edit</code> , <code>remove</code> , <code>on</code> , <code>off</code> , <code>start</code> , <code>stop</code> e <code>gui</code>
Interface	<code>gui.py</code>	Janela Tkinter para criar, editar, apagar e ativar lembretes, e para iniciar ou parar o serviço
Interface	<code>tray.py</code>	Ícone na bandeja com o menu “Abrir” e “Sair”
Lógica	<code>app.py</code>	Liga as peças: arranque, paragem e

Camada	Módulo	Função
		decisão sobre o que fazer quando um lembrete dispara
Lógica	<code>scheduler.py</code>	Traduz cada lembrete num gatilho do APScheduler
Lógica	<code>notifications.py</code>	Notificação do sistema e pop-up no canto do ecrã
Dados	<code>models.py</code>	Estrutura Lembrete, o contrato partilhado por todos os módulos
Dados	<code>storage.py</code>	Leitura e escrita na base de dados SQLite

Os diagramas da arquitetura, da divisão de responsabilidades e do ciclo de vida do processo estão no repositório: [arquitetura](#), [responsabilidades](#) e [ciclo de vida](#).

6.1 Como as peças funcionam em conjunto

O programa tem dois tipos de processo. O serviço residente, lançado por `rememberme start` ou pelo duplo clique no executável, mantém o ícone na bandeja e o agendador ligados. Os comandos da linha de comandos e a interface gráfica são processos separados, que escrevem na base de dados e terminam (ou, no caso da GUI, ficam abertos até o utilizador a fechar).

Dentro do serviço há duas threads (subprocessos). A principal pertence ao ícone da bandeja, que a bloqueia até o programa encerrar; a segunda é a do agendador, que dispara os lembretes.

Os processos não falam diretamente uns com os outros. Comunicam através de uma pequena tabela estado na base de dados. O comando `stop` escreve lá um pedido de paragem, e o serviço consulta essa tabela de cinco em cinco segundos (intervalo definido arbitrariamente pela constante `INTERVALO_VIGIA`) e encerra por iniciativa própria. O “Sair” do menu e o `stop` passam pela mesma função, `encerrar()`, para que nenhum dos dois caminhos se esqueça de limpar o estado. O processo nunca é terminado à força porque, em Windows, isso deixa o ícone órfão na bandeja até alguém lhe passar o rato por cima.

6.2 Dados

A base de dados fica fora do repositório, na pasta de dados do utilizador (%APPDATA%\RememberME\rememberme.db). Versioná-la gerava conflitos a cada execução e expunha os dados de quem usa o programa.

Cada lembrete guarda o texto, a hora fixa ou a janela com o intervalo, os dias da semana, se está ativo e o tipo de ação. Não existe um campo “tipo”: um lembrete com hora preenchida é de hora fixa, e um sem hora é recorrente. Guardar o tipo e os campos que o determinam criaria duas fontes para a mesma informação.

6.3 Decisões de desenho

As decisões técnicas foram registadas à medida que foram tomadas, com data e motivo, em `deciso.es.md`. São nove, entre elas o layout `src/` do código, a base de dados fora do repositório, a escolha do APScheduler, o isolamento de cada biblioteca num único ficheiro e o uso de Pull Request apenas na passagem de `develop` para `master`.

7. Organização e distribuição do trabalho

A divisão foi pensada com dois critérios. O primeiro era evitar conflitos de merge: cada pessoa ficou dona de ficheiros diferentes, de modo a não estarem dois elementos a editar o mesmo código ao mesmo tempo. O segundo era a diferença de experiência dentro do grupo. O Renato tinha mais prática anterior e ficou com a coordenação e com os módulos onde o risco técnico era maior (o agendador, a bandeja e a ligação entre os dois). Para o Felipe e a Niley, o Renato preparou primeiro um esqueleto com as funções já declaradas e documentadas, para que pudessem preencher código com um contrato fixo em vez de desenhar módulos do zero.

O trabalho foi organizado em quatro fases, registadas no [quadro de tarefas](#) com responsável, estado, data prevista e dependências:

Fase	Conteúdo	Período
0 – Capacitação	Cada elemento estudou a biblioteca do seu módulo com um pequeno protótipo; comparação dos motores de agendamento	2 a 7 de setembro
1 – Fundação	<code>.gitignore</code> , <code>pyproject.toml</code> , contrato de dados e esqueleto dos módulos	3 a 9 de setembro
2 – MVP	Base de dados, agendador, bandeja, linha de comandos e notificações	7 a 22 de setembro

Fase	Conteúdo	Período
3 – Validação e incrementais	Testes, empacotamento, tipos de ação, arranque automático e interface gráfica	21 a 28 de setembro

A Fase 0 foi incluída porque ninguém no grupo conhecia as bibliotecas escolhidas. Tratar o estudo como parte do plano, e não como algo prévio, permitiu que os problemas técnicos aparecessem na primeira semana em vez de na integração final.

As integrações em develop aconteceram em dias de aula, com os três presentes, e a revisão do código foi feita em conjunto durante essas sessões.

8. Descrição das funcionalidades implementadas

8.1 Lembretes a hora fixa (RF01)

O utilizador cria um lembrete para uma hora do dia, opcionalmente limitado a certos dias da semana. Exemplo: `rememberme add "Fazer a chamada" --as 09:15 --dias seg,qua,sex`.

8.2 Lembretes recorrentes numa janela (RF02)

O lembrete repete-se de X em X minutos dentro de um intervalo de horário: `rememberme add "Fechar sessão" --entre 08:00-14:00 --a-cada 60`.

8.3 Gestão de lembretes (RF06)

Na linha de comandos, `list` mostra os lembretes numa tabela, `edit` altera qualquer campo, `remove` apaga, e `on` e `off` ativam ou suspendem um lembrete sem o apagar. Os erros de utilização (hora inválida, opções incompatíveis, identificador inexistente) têm mensagens próprias. A interface gráfica faz o mesmo através de um formulário e de uma lista.

8.4 Serviço na bandeja (RF03)

`rememberme start` lança o serviço em segundo plano e devolve a consola. O ícone aparece na bandeja; um clique abre a janela de gestão e o menu tem “Abrir” e “Sair”. O serviço verifica se já existe outro a correr antes de arrancar, para não duplicar ícones e avisos.

8.5 Persistência local (RF04)

Os lembretes sobrevivem ao fecho do programa e ao reinício do computador, guardados em SQLite na pasta do utilizador.

8.6 Tipos de aviso (RF07, parcial)

Quando um lembrete dispara, o programa pode mostrar uma notificação do sistema, abrir um pop-up no canto do ecrã, tocar um som ou abrir uma aplicação ou página web. Na versão entregue, os lembretes criados pela linha de comandos usam a notificação do sistema e os criados pela GUI usam o pop-up. As restantes ações ainda não estão acessíveis ao utilizador.

8.7 Interface gráfica (RF10, parcial)

Janela em Tkinter para criar, editar, apagar e ativar lembretes e para iniciar ou parar o serviço. Só pode estar aberta uma de cada vez.

8.8 Distribuição e arranque automático (RF08)

Há duas formas de instalar:

- RememberME-Portable-v1.0.exe, um único ficheiro que corre sem instalação;
- RememberME-Setup-v1.0.exe, um instalador que coloca o programa em C:\Program Files\RememberME, cria atalhos e, se o utilizador quiser, põe o RememberME a arrancar com o Windows.

Ambos estão na [release v1.0](#) e são gerados pelo script `build.ps1`.

9. Estratégia de utilização de Git e GitHub

9.1 Branches

O master recebe apenas versões estáveis, e o develop é onde o trabalho se junta. O trabalho diário foi feito em branches próprias, com a convenção `<tipo>/<nome>-<descrição>`, por exemplo `feat/niley-notificacoes`, `feat/felipe-cli-comando-add` ou `feat/renato-criar-agendador`. Os tipos usados foram `feat`, `fix`, `docs`, `test` e `chore`.

9.2 Commits

A convenção era `<tipo>`: `<descrição no infinitivo>`, como em `feat`: adicionar comando remove na CLI. Ao todo há 119 commits dos três elementos, feitos ao longo de todo o projeto e não concentrados no fim.

9.3 Pull Requests

Ficou decidido (decisão D8) que a passagem de `develop` para `master` seria sempre por Pull Request, uma por entrega, e que o trabalho diário entraria em `develop` por merge direto, já que cada pessoa trabalhava em ficheiros diferentes. Na prática, o Felipe e a Niley abriram também Pull Requests das suas branches para `develop`. Foram abertos [11 Pull Requests](#), dos quais 10 foram integrados; por exemplo, o [#5](#) (arranque automático), o [#7](#) (notificação e som), o [#10](#) (notificações) e o [#11](#) (versão 1.0).

9.4 Revisão de código

As revisões aconteceram presencialmente, nas aulas, com o grupo junto ao ecrã. Resultaram em correções diretas, não em comentários registados no GitHub, o que é uma fragilidade que assumimos (ver secção 13).

9.5 Versões

Cada entrega ficou marcada com uma etiqueta Git. As versões principais foram publicadas como releases no GitHub: a [v0.5, protótipo funcional](#), a 22 de setembro, e a [v1.0](#), a 28 de setembro.

9.6 Documentação viva

O quadro de tarefas e o registo de decisões ficam na pasta *docs/* e foram atualizados ao longo do projeto. O estado de qualquer um deles numa data passada recupera-se pelas etiquetas.

Não houve integração contínua. Foi uma decisão consciente, tendo em conta o tempo e a experiência do grupo, e as integrações foram feitas em momentos agendados.

10. Testes realizados

Os testes foram manuais, seguindo o [plano de testes](#) da Etapa 4. Cada teste tem pré-condições, passos, resultado esperado e resultado obtido, e foi gravado em vídeo. Os vídeos estão na [pasta de vídeos do repositório](#).

Requisito	Testes	O que foi verificado	Resultado
RF01, hora fixa	T01 a T04	Criação, erros de validação, disparo à hora marcada	4 de 4 passaram
RF02, janela	T05 a T09	Criação, combinações de opções inválidas, disparo dentro da janela	5 de 5 passaram
RF03, bandeja	T10 a T12	Ícone ao arrancar, “Sair” pelo menu, paragem pelo stop	3 de 3 passaram
RF04, persistência	T13 e T14	Lembretes mantêm-se	2 de 2 passaram

Requisito	Testes	O que foi verificado	Resultado
		após reinício; ficheiro fora do repositório	
RF06, gestão	T15 a T22	Listar, editar, ativar, desativar e remover, incluindo identificadores inexistentes	8 de 8 passaram

Os 22 testes passaram. Ficaram de fora os requisitos que não estavam acessíveis de ponta a ponta ao utilizador (RF05, RF07, RF09 e RF10) e o RF08, que depende do instalador. A interface gráfica e o instalador foram verificados de forma exploratória durante o desenvolvimento, sem plano formal. Testes foram realizados à GUI e ao instalador depois da Etapa 4, porém, informalmente e sem registo.

Não há testes automatizados no repositório, no entanto.

11. Dificuldades encontradas

Começou pela curva de aprendizagem. Nenhum elemento conhecia as bibliotecas escolhidas, e dois dos três estavam ainda a aprender Python e Git. O estudo das bibliotecas consumiu tempo que, num grupo mais experiente, iria para funcionalidades.

Depois, o modelo de threads. O ícone da bandeja ocupa a thread principal e bloqueia-a, o agendador precisa de correr ao mesmo tempo, e os comandos da linha de comandos são outro processo. Fazer estas três peças conviverem, e conseguir parar o serviço de fora sem o matar, foi o problema de desenho mais delicado do projeto.

O empacotamento trouxe problemas que não se viam em desenvolvimento. O executável de ficheiro único extrai-se para uma pasta temporária e, quando relançava um segundo processo (para o serviço ou para a janela), esse processo herdava a pasta do primeiro e perdia-a quando o primeiro fechava. O p1yer também carrega os seus componentes de forma dinâmica, o que o PyInstaller não deteta sozinho.

Houve ainda dificuldades de colaboração no Git: conflitos de merge no `storage.py` e no quadro de tarefas, alguns commits duplicados ao integrar branches, uma coluna da base de dados declarada duas vezes depois de um merge, e ficheiros criados por engano por comandos de terminal que chegaram a ser commitados.

Por fim, o tempo. As 32 horas de aula para cinco etapas obrigaram a escolher o que ficava de fora, e a interface gráfica abria várias janelas e processos repetidos nas primeiras versões.

12. Soluções adotadas

Para a aprendizagem, cada elemento fez um pequeno protótipo (spike) com a biblioteca do seu módulo antes de escrever código definitivo, e o esqueleto com funções já declaradas deu ao Felipe e ao Niley um ponto de partida concreto, o que agilizou o desenvolvimento já que os contratos entre as funções já estavam preestabelecidos e, ainda, evitou maiores problemas de conflito ao dar uma visão clara de onde cada membro da equipa deveria realizar a implementação. A escolha do agendador foi feita comparando as duas opções com código a correr e com os dados recolhidos dele.

O modelo de threads foi validado cedo, na Fase 1, com um esqueleto que já mostrava o ícone e disparava uma notificação real. A comunicação entre processos passou pela tabela estado da base de dados, que o serviço já consultava de qualquer forma, e todas as saídas passaram por uma única função de encerramento.

No empacotamento, os processos relançados passaram a pedir uma extração própria (através da variável `PYINSTALLER_RESET_ENVIRONMENT`) e o módulo do `p1yer` para Windows foi declarado explicitamente no ficheiro de configuração do `PyInstaller`.

A duplicação de processos foi resolvida com duas proteções: o serviço guarda o seu identificador de processo num ficheiro e não arranca se já houver outro ativo, e a interface gráfica recusa abrir uma segunda janela. Curiosamente, esta proteção tinha sido retirada do plano na Etapa 2 por não ser pedida por nenhum requisito. O uso real mostrou que fazia falta e ela voltou, numa versão mais simples.

Os conflitos de Git foram resolvidos à mão, nas sessões de integração, e as correções ficaram registadas em commits próprios (`fix` e `chore`). A divisão por ficheiros manteve, de facto, os conflitos raros.

Quanto ao tempo, a ordem de prioridades definida na Etapa 2 decidiu o que cairia primeiro: o RF09 e parte do RF07 e do RF10 ficaram por fazer, e o núcleo do MVP foi testado e empacotado.

13. Limitações

- O RF05 não foi implementado: a base de dados começa vazia, sem os lembretes administrativos pré-carregados.
- O RF07 está incompleto. A lógica dos quatro tipos de aviso existe, mas o utilizador não consegue escolher entre eles.
- O menu da bandeja não mostra os lembretes ativos, como o RF03 previa.
- Na análise final do código identificámos limitações no agendamento recorrente: intervalos superiores a 60 minutos (por exemplo, 90 ou 120) não são aceites pelo agendador e o lembrete é ignorado sem aviso ao arrancar; intervalos que não dividem a hora, como 45 minutos, dão disparos irregulares (08:00, 08:45, 09:00, 09:45); a janela só considera as horas, pelo que uma janela das 08:30 às

14:00 começa às 08:00 e pode disparar ainda durante as 14h; e janelas que atravessam a meia-noite não funcionam.

- A interface gráfica não atualiza a lista sozinha; é preciso carregar em “Atualizar lista”.
- Há apenas executável para Windows de 64 bits, e a GUI e o som usam funções exclusivas do Windows.
- O executável de ficheiro único arranca mais devagar e alguns antivírus desconfiam dele, por não ser assinado.
- Não há testes automatizados, e as revisões de código não ficaram registadas no GitHub.
- Um executável de cerca de 19 MB chegou a ser commitado no repositório, o que aumenta o tamanho do histórico.
- As versões empacotadas para Windows silenciam o uso do executável pela linha de comando.

14. Melhorias futuras

- Completar o RF07, com uma opção para escolher o tipo de aviso na linha de comandos e mais opções na GUI.
- Carregar os lembretes administrativos habituais na primeira execução (RF05).
- Acrescentar ao menu da bandeja a lista dos lembretes ativos.
- Corrigir a tradução das janelas e dos intervalos no agendador, usando um gatilho de intervalo limitado pela janela em vez de uma expressão de minutos.
- Criar testes automatizados para a base de dados e para o agendador, e corrê-los no GitHub a cada Pull Request.
- Implementar o RF09 com confirmação e registo, como exige o RNF05.
- Testar e empacotar para macOS e Linux, substituindo as partes que só funcionam em Windows.
- Assinar o executável e retirar os binários do histórico do repositório, publicando-os apenas nas releases.

15. Conclusão

O grupo entregou uma aplicação que resolve o problema de partida. O formador instala o RememberME, cria os seus lembretes pela janela ou pela linha de comandos e recebe os avisos à hora certa, sem ter o programa à frente. O núcleo definido na Etapa 1 ficou implementado quase na totalidade, com a exceção dos lembretes pré-carregados, e foi testado com 22 casos gravados em vídeo. Dos requisitos incrementais, entraram o arranque automático e versões parciais da interface gráfica e dos tipos de aviso.

Do ponto de vista do processo, o projeto obrigou três pessoas com níveis de experiência muito diferentes a trabalhar no mesmo repositório durante aproximadamente um mês. O histórico mostra os três a contribuir ao longo de todo esse tempo, com branches por funcionalidade, Pull Requests e versões etiquetadas.

Também mostra os tropeços: conflitos, commits duplicados, ficheiros enviados por engano. Também foi com eles que aprendemos a usar a ferramenta.

A decisão que mais pesou no resultado foi separar cedo o que era obrigatório do que era opcional. Quando o tempo apertou, sabíamos o que podia cair. Mesmo assim, nos lançamos para tentar entregar o opcional.

16. Apreciação do contributo de cada elemento

Os números abaixo vêm do histórico do repositório e somam as várias contas de cada pessoa. O número de commits mede a frequência, não o esforço: um commit pode ser uma linha ou uma funcionalidade inteira.

Elemento	Commits	Pull Requests abertos
Renato Moraes	52	4
Niley Barros	42	1
Felipe Ribeiro	25	6

16.1 Felipe Ribeiro

Implementou os comandos da linha de comandos (add, list, edit, remove, on e off) e documentou os respetivos testes. Preparou o arranque automático no Windows, trabalhou na notificação com som e criou a interface gráfica em Tkinter, incluindo a correção que impedia a abertura de várias janelas e processos. Tratou do executável portátil e do instalador.

Apreciação pessoal sobre o próprio trabalho e dos colegas sobre o contributo dele:

No RememberME, concentrei o meu contributo nas partes que exigiam maior rigor técnico na interação com o utilizador e na integração entre módulos. Implementei os comandos da linha de comandos (add, list, edit, remove, on e off), garantindo validação robusta, mensagens de erro consistentes e compatibilidade total com o modelo de dados definido na Etapa 1.

Na interface gráfica, desenvolvi a janela em Tkinter e corriji problemas críticos relacionados com a abertura simultânea de múltiplas instâncias, que afetavam a estabilidade do serviço residente. Trabalhei também na integração das notificações com som, enfrentando dificuldades específicas no empacotamento com PyInstaller, sobretudo na carga dinâmica de componentes do plyer e no comportamento distinto entre o ambiente de desenvolvimento e o executável final.

Além disso, preparei o executável portátil, o instalador e o arranque automático no Windows, tarefas que exigiram testes repetidos e ajustes finos devido às diferenças entre processos relançados e o isolamento do ambiente de execução.

Apesar das dificuldades técnicas — especialmente na GUI, no empacotamento e na compatibilidade das notificações — considero que o meu contributo foi decisivo para a entrega de funcionalidades acessíveis ao utilizador e para a estabilidade da versão final. O projeto permitiu-me consolidar competências práticas em Python, Tkinter, empacotamento, gestão de processos e colaboração estruturada em Git.

O trabalho em equipa decorreu de forma coordenada e contínua, com cada elemento a acompanhar o progresso das etapas e a contribuir para manter o projeto alinhado com os requisitos definidos. As integrações presenciais facilitaram a resolução de pontos técnicos e permitiram ajustar o desenvolvimento sempre que necessário.

16.2 Niley Barros

Fez o estudo inicial do p1yer e a primeira versão das notificações. Implementou a base do RF07: os campos de ação no modelo e na base de dados e a lógica que decide o que acontece quando um lembrete dispara. Executou e registou os 22 testes da Etapa 4, com os vídeos correspondentes, e manteve o quadro de tarefas atualizado.

16.3 Renato Moraes

Coordenou o grupo e escreveu a maior parte da documentação das etapas, incluindo o plano de desenvolvimento e o registo de decisões. Montou a estrutura do projeto e o esqueleto dos módulos, e implementou o modelo de dados, a base de dados, o agendador, a bandeja e a ligação entre eles, bem como o arranque em segundo plano. Organizou as versões estáveis por meio de Pull Requests e das releases.

Apreciação pessoal sobre o próprio trabalho e dos colegas sobre o contributo dele:

Meu desafio central foi equilibrar e distribuir continuamente as tarefas técnicas e administrativas do projecto de acordo com a perspetiva e avaliação do desempenho de cada membro da equipa. Para isso, após a escolha do tema, problema e objectivos, estudei, organizei e testei preliminarmente as bibliotecas seleccionadas. Na sequência, escrevi a base fundamental dos ficheiros e funções, estabelecendo uma base concreta para a equipa atuar. Dessa forma pude tentar garantir maior taxa de sucesso na experiência de desenvolvimento.

Após a entrega do esqueleto e da documentação inicial, me ocupei de gerir a forma como as entregas eram realizadas, definindo escopo, nomenclaturas e fluxo de trabalho no git. A decisão de proteger versões estáveis no master de eventuais merges, definitivamente foi acertada e evitou problemas maiores nas entregas intermediárias.

As nomenclaturas definidas para os branches facilitou a observação do escopo e evolução do trabalho de cada membro da equipa.

Durante o desenvolvimento, além de me concentrar na parte de threads (subprocessos), que não foi abordado no curso (e que eu também não tinha experiência prática, mas tinha conhecimento teórico) e no o agendador (que possuía grande relevância nuclear do projecto), também me atentei para eventuais problemas de integração de códigos e defeitos não previstos, antes das entregas.

Em relação aos demais membros da equipa, o desenvolvimento fluiu naturalmente, com eventuais problemas técnicos que foram sanados à medida em que o projecto de desenvolveu. De modo geral, todos foram solícitos às tarefas quando expunha as questões e estratégias elaboradas, destacando a proatividade do Felipe em questões e soluções que eu mesmo já havia desistido. No entanto, relacionado ao desempenho colaborativo global, acredito que um melhor protocolo de comunicação favoreceria melhor assertividade e sucesso na apresentação.

16.4 Apreciação global

Parágrafo acordado entre os três membros da equipa sobre o equilíbrio da divisão, o que correu bem e o que fariam de forma diferente:

A equipa manteve uma colaboração estável ao longo do projeto, o progresso das etapas e a contribuir dentro das respectivas áreas de responsabilidade. As integrações presenciais permitiram alinhar decisões técnicas, resolver pontos críticos e garantir que o núcleo definido na Etapa 1 avançava de forma consistente até à versão final.